Technical report · Version 1.0 · October 2026

# OrbiBot V1: A Mecanum Mobile Robot with Microcontroller-Level Real-Time Control, a ROS 2 Navigation Stack, and a Tool-Using LLM Agent

Design and Bench Evaluation

**Jewel Nath** Independent researcher, Dhaka, Bangladesh

ORCID 0009-0002-6609-6378 · Code and data: https://github.com/jewel-nath/orbibot-v1

**Abstract**

OrbiBot V1 is a four-wheel mecanum robot built by hand from a welded stainless-steel frame, a Raspberry Pi 5 running ROS 2 Jazzy, and an STM32F103 motor board running new open-source firmware. Its design rests on one split: every task with a deadline (a 100 Hz PID loop per wheel, encoder counting, odometry, and a 2 s command timeout) runs on the microcontroller under FreeRTOS, and everything that needs a map, a model or a network runs on the host.

On top of this base the robot maps with slam_toolbox, fuses wheel odometry and a gyro in an extended Kalman filter, runs Nav2 with the MPPI controller in its omnidirectional mode, is driven from a browser dashboard, and exposes 58 robot tools to large language models through an HTTP interface, a terminal client and a Model Context Protocol server, with a safety layer on its direct motion tools.

The bench tests, made with the wheels off the ground, show chassis velocity within 1.3–9.3 % of the command, motors stopped between 1.82 and 2.12 s after the last command against a 2 s setting, a velocity clamp that held a 3 m/s command to 1.02 m/s, four motors within 0.1 % of each other at the speed limit, and a serial link that passed malformed-input tests after one critical fix. Mapping, navigation, the dashboard and the agent were run on the robot but not measured; the report states each such limit.

**Keywords:** mobile robots, mecanum wheels, ROS 2, embedded control, FreeRTOS, SLAM, Nav2, large language models, tool use, Model Context Protocol.

## 1 Introduction

Hobby-scale mobile robots can now run the same navigation software as research platforms. ROS 2 [1], Nav2 [2] and slam_toolbox [3] run on a single-board computer, and large language models (LLMs) can call software tools well enough to drive a robot through them [4].

Putting these on one low-cost robot raises a practical problem. The host that runs mapping, planning, a web server and an LLM client is not real-time, yet the wheels need a steady control loop and a reliable way to stop. OrbiBot V1 answers that with a strict division of labour between two computers. This report

describes the result as built, and contributes:

1. a two-computer architecture in which all deadline-bound work lives on the microcontroller, designed so that load on the host cannot disturb the wheels (Section 3);
2. new firmware for a commercial STM32 motor board, replacing the vendor's closed original, with a documented binary protocol and layered safety behaviour (Section 5);
3. an integration of the ROS 2 estimation, mapping and navigation stack for a mecanum base (Section 7);
4. an LLM agent that exposes 58 robot tools to several model providers and through the Model Context Protocol (MCP) [5], with a safety manager in front of its direct motion tools (Section 9);
5. bench measurements of the firmware, and a plain account of what has not been measured (Sections 10 and 11).

OrbiBot's own code, documentation and test reports are open source under Apache-2.0 [6]. The third-party parts keep their own licences, and the serial driver derived from the board vendor's library has no stated licence.

**Results at a glance**

| What | Result | Status |
|---|---|---|
| Wheel directions | all four motion patterns match the kinematics | measured |
| Velocity tracking | 98.7, 102.7, 90.7 and 104.1 % of the command (forward, backward, sideways, turning) | measured |
| Motors at 1.0 m/s | 1001.7–1002.5 mm/s, 0.1 % apart | measured |
| 2 s command timeout | motors stopped between 1.82 and 2.12 s | measured |
| 1.0 m/s clamp | 1023 mm/s peak on a 3 m/s command | measured |
| Serial link | 270 frames/s; 1083 commands in 3.3 s, no errors | measured |
| Truncated frames | a critical fault found and fixed | measured |
| Accelerometer at rest | 0.9995 g | measured |
| Mapping, map driving, dashboard | worked on the robot, on video | run, not measured |
| Nav2, LLM agent | worked on the robot, status notes only | run, not measured |
| Floor drift, Nav2 success, agent success, host load | – | not measured |

All measurements were made on the bench with the wheels off the ground, on 2026-04-19, 2026-04-28 and 2026-08-03.

## 2 Related Work

**Open mobile platforms.** Open ROS 2 mecanum robots such as LGDXRobot2 [7] pair a microcontroller for motor control with a Linux host, as OrbiBot does; their published MPPI weights and IMU calibration

informed this build. Mecanum kinematics for feedback control go back to Muir and Neuman [8].

**Estimation, mapping and navigation.** The robot uses established components: the generalised EKF of robot_localization [9], Madgwick's orientation filter [10], slam_toolbox for graph-based mapping [3], AMCL [11], and Nav2 [2] with the model-predictive path integral (MPPI) controller [12]. Exploration follows the frontier method of Yamauchi [13], as implemented in m-explore-ros2 [14].

**Language models and robots.** SayCan grounds an LLM's plans in a robot's skills [15]; Code as Policies has the model write robot programs [16]; Vemprala et al. set out design principles for prompting ChatGPT to control robots through high-level functions [4]. Toolformer showed that models can learn when to call tools [17].

OrbiBot's agent is closer to the function-library approach: it gives the model a fixed set of typed tools and checks motion outside the model. It adds the MCP [5], so that general-purpose AI clients can operate the robot through the same tools.

## 3 System Overview

Fig. 1 shows the whole system. A Raspberry Pi 5 runs Ubuntu 24.04 and ROS 2 Jazzy, with the LiDAR and the depth camera on its USB ports. A Yahboom ROSMaster V3.0 board, with an STM32F103RCT6 (Cortex-M3 at 72 MHz), drives the four motors and reads the encoders and the IMU. The two talk over a USB serial link at 115200 baud.

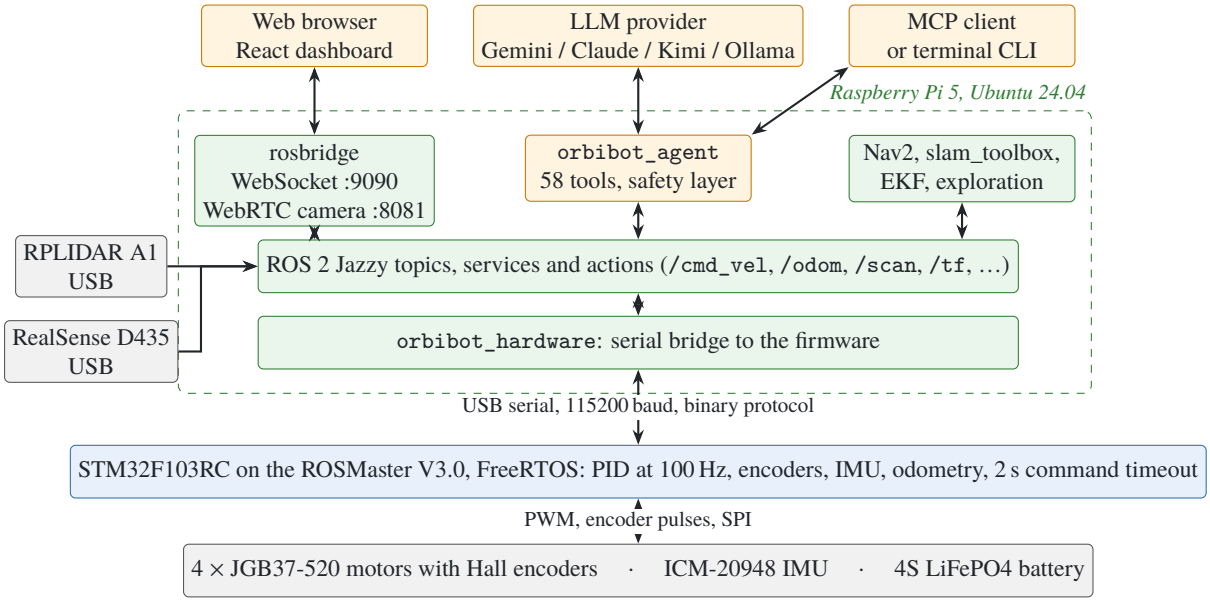



**Figure 1:** System architecture. The STM32 (blue) closes the fast loops; the Raspberry Pi (green) runs ROS 2, mapping, navigation and the interfaces (orange). Every interface reaches the wheels only through ROS 2 topics and the serial bridge.

The rule behind the split is simple: *anything with a deadline lives on the microcontroller*. The PID loops, encoder counting, odometry and the command timeout cannot be pre-empted by anything on the host. By

design, then, a busy Pi, a heavy dashboard or a slow LLM response should not make the wheels stutter; this has not yet been tested under load. The host never has to be real-time, and it can be replaced without touching the firmware.

The robot was built over fifteen months, from June 2025 to August 2026 (timeline in Appendix, Fig. 14).

## 4 Hardware

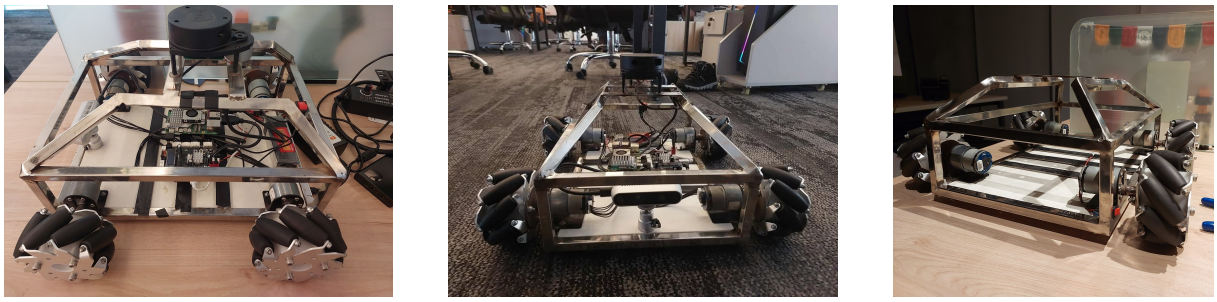


**Figure 2:** OrbiBot V1. Left: the Raspberry Pi 5 build, with the RPLIDAR A1 on top and the Pi above the motor board (September 2025). Middle: after the rebuild, with the RealSense D435 at the front (November 2025). Right: the welded stainless-steel frame, stripped for the rebuild, with its four JGB37-520 motors and encoders (November 2025).

The frame is welded stainless steel, $350 \times 250$ mm in the robot model, on four 100 mm steel mecanum wheels in an ABBA layout (Fig. 2). Each wheel is driven by a JGB37-520 geared DC motor (30:1, 333 rpm at 12 V with no load) with a Hall-effect quadrature encoder giving 1320 counts per wheel turn. The robot model totals about 8 kg, 6 kg of it the frame link; the robot was not weighed.

Table 1 lists the parts and Fig. 3 the power and data paths.

**Table 1:** Main parts: datasheet values, with firmware settings marked (f) and measurements marked (m).

| Part | Key figures |
|---|---|
| **Host** | Raspberry Pi 5, 8 GB; 4 × Cortex-A76 at 2.4 GHz; 49.4 °C at idle (m) |
| **Motor board** | Yahboom ROSMaster V3.0, STM32F103RCT6, 256 KB flash, 48 KB RAM; 4 AM2857 H-bridges, PWM at 20 kHz (f) |
| **Motors** | 4 × JGB37-520, 12 V, 30:1, 333 rpm, 3.3 kg·cm rated |
| **Wheels** | 4 × 100 mm steel mecanum, 45° rollers |
| **IMU** | ICM-20948 on SPI; gyro ±500 °/s and accelerometer ±2 g ranges (f); magnetometer unused |
| **LiDAR** | RPLIDAR A1, 360°, 0.15–12 m, 5.5 Hz nominal |
| **Depth camera** | Intel RealSense D435 (on a USB 2 link on V1) |
| **Battery** | 4S LiFePO4, 12.8 V nominal, 15.5 Ah, about 198 Wh; 100 A BMS |

Two hardware facts shaped the software.

- **The depth camera ran on a USB 2 link**, at which the D435 cannot deliver depth and colour together at a useful resolution. On V1 the camera launch is off by default, and the dashboard uses

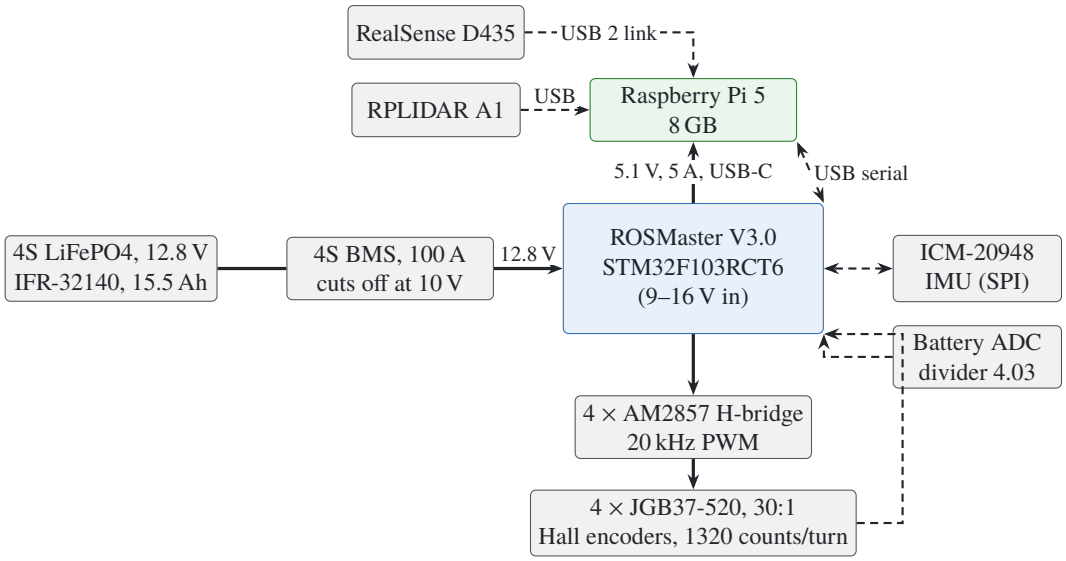



**Figure 3:** Power (solid lines) and data (dashed). The motor board takes the pack voltage and supplies the Pi over USB-C; the LiDAR and the camera hang off the Pi's USB ports.

only its colour stream.

- **The IMU's magnetometer was removed from use** on 2026-04-28, because indoors the steel frame, motors and battery currents distorted the field enough to make heading worse, not better. The robot therefore has no absolute heading of its own (Section 7).

## 5 Firmware

The board ships with closed-source firmware. OrbiBot replaces it with new firmware written in C with PlatformIO, on FreeRTOS [18]. It uses 28.4 KB of flash (10.9 %) and 19.9 KB of RAM (40.4 %), measured with GCC 12.3, so memory is not the limit; CPU time on a Cortex-M3 without a floating-point unit is.

### 5.1 Tasks

Four tasks share the processor (Fig. 4):

- the **control task**, at 100 Hz with the highest priority, reads the four encoders, runs one PID loop per wheel, writes the PWM and integrates odometry;
- the **IMU task** reads the gyro and accelerometer at 50 Hz;
- the **communication task** ticks every millisecond, parses incoming commands and sends five report types, each at 50 Hz;
- the **monitoring task**, at 1 Hz, reads the battery, drives the buzzer and feeds a 5 s independent watchdog only when every other task has shown it is alive.

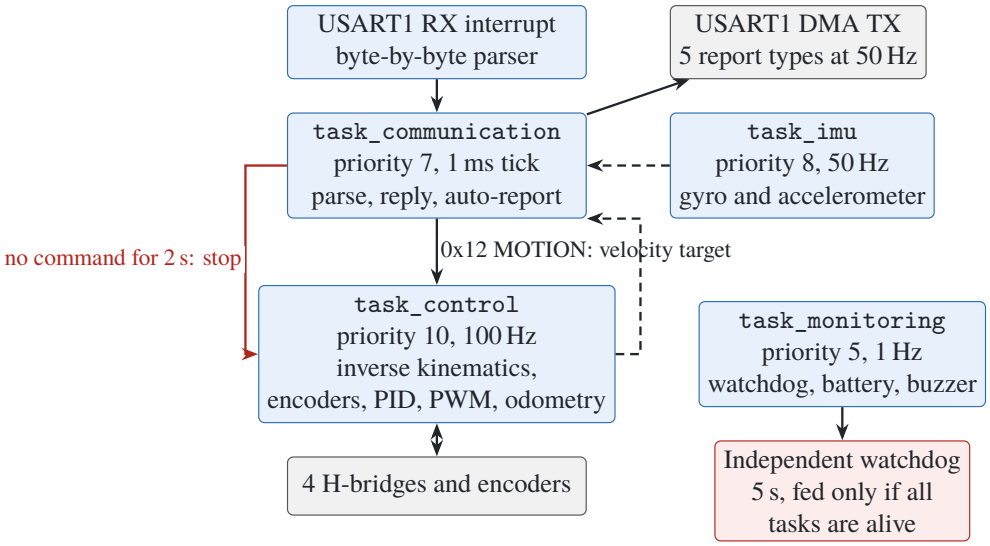



**Figure 4:** Firmware tasks and the path of a motion command. If no command arrives for 2 s, the control task stops the motors.

## 5.2 Speed control

Each wheel has an incremental PID controller:

$$\Delta u_k = K_p(e_k - e_{k-1}) + \tfrac{K_i}{2}(e_k + e_{k-1}) + K_d(e_k - 2e_{k-1} + e_{k-2}), \qquad u_k = u_{k-1} + \Delta u_k, \tag{1}$$

with default gains $K_p = 0.8$, $K_i = 0.06$, $K_d = 0.5$, which can be changed over the serial link and stored in flash. A conditional clamp at $\pm 4000$ stops the integral term from winding up when the output is saturated in the direction of the error.

The H-bridges have a dead band, which the firmware crosses by adding a fixed offset of 1600 (of 3600) to any non-zero PWM. Section 11 shows what this costs at very low speed.

## 5.3 Protocol

The host and the board exchange binary frames compatible with the vendor's protocol, so that existing tools still work (frame format in Appendix, Fig. 15). The host sends motion commands $(v_x, v_y, \omega)$ as 16-bit integers, PID gains, resets, buzzer commands and version requests. The firmware reports speed and battery voltage, raw IMU data, attitude, the four encoder counts and odometry. At 115200 baud the five report streams use about a third of the link.

## 5.4 Safety in the firmware

The firmware enforces its own limits whatever the host sends:

- it clamps commands to 1.0 m/s and 2.0 rad/s;
- it stops the motors 2 s after the last command, so a crashed host or a closed browser tab cannot leave

the robot driving;

- a reset-state command stops the motors at once;
- it warns at 11.0 V and locks the motors out at 10.5 V;
- the watchdog resets the processor if a task hangs.

These limits apply to every source of commands, including those that bypass the agent's safety manager (Section 9). Fig. 10 places them among the other safety layers.

## 6 Kinematics and Odometry

With $x$ forward, $y$ to the left, wheel radius $r = 50\,\text{mm}$, half-wheelbase $l_x = 130\,\text{mm}$ and half-track $l_y = 180\,\text{mm}$ (Fig. 5), the firmware converts a body velocity into wheel speeds with the standard inverse kinematics of a mecanum base [8]:

$$\begin{bmatrix} \omega_{FL} \\ \omega_{FR} \\ \omega_{BL} \\ \omega_{BR} \end{bmatrix} = \frac{1}{r} \begin{bmatrix} 1 & -1 & -(l_x + l_y) \\ 1 & 1 & l_x + l_y \\ 1 & 1 & -(l_x + l_y) \\ 1 & -1 & l_x + l_y \end{bmatrix} \begin{bmatrix} v_x \\ v_y \\ \omega \end{bmatrix}. \tag{2}$$

Odometry uses the matching forward kinematics, integrated at 100 Hz:

$$v_x = \tfrac{r}{4}\textstyle\sum_i \omega_i, \qquad v_y = \tfrac{r}{4}(-\omega_{FL}+\omega_{FR}+\omega_{BL}-\omega_{BR}), \qquad \omega = \tfrac{r}{4(l_x+l_y)}(-\omega_{FL}+\omega_{FR}-\omega_{BL}+\omega_{BR}). \tag{3}$$

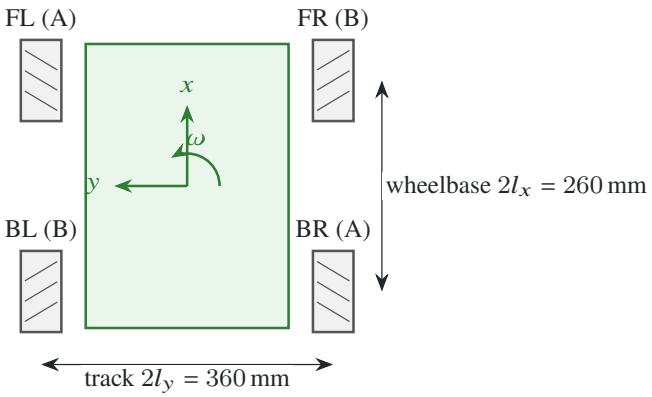



**Figure 5:** Top view of the base, to scale. The wheels stand outside the frame (350 × 250 mm in the robot model); roller directions are drawn schematically, as seen from above.

## 7 ROS 2 Software

Twelve ROS 2 packages run on the host: the serial bridge, custom messages, the robot model, sensors, localisation, SLAM, navigation, exploration, launch files, the web dashboard, the agent, and a gamepad driver now switched off. Fig. 6 shows how the main ones connect.

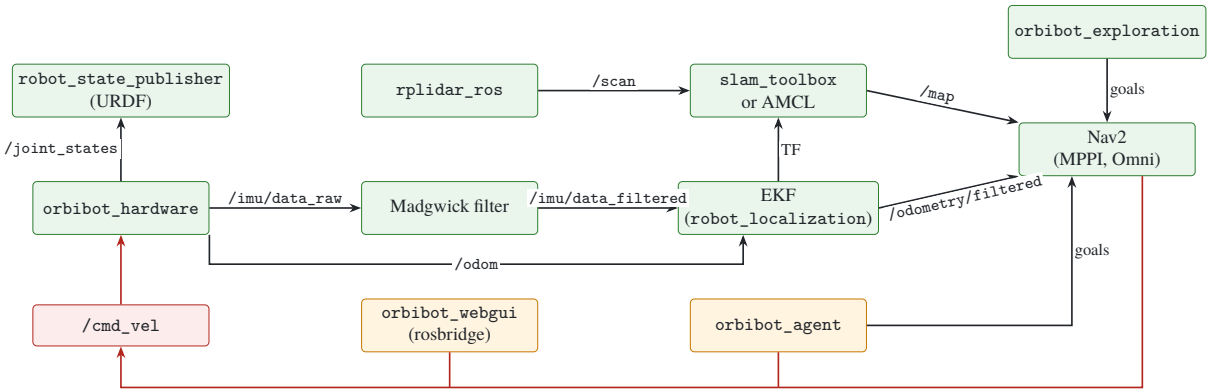



**Figure 6:** Main ROS 2 packages and topics. The dashboard, the agent and Nav2 write `/cmd_vel` (red), and the switched-off gamepad driver is still wired to it; nothing arbitrates between the four (Section 11).

**The bridge.** The serial bridge, `orbibot_hardware`, turns the firmware's reports into the topics `/odom`, `/imu/data_raw` and `/joint_states` and into status topics, and turns `/cmd_vel` into motion frames. Its driver started from the vendor's Python library.

**Estimation.** A Madgwick filter [10] runs in six-axis mode, without the magnetometer, and an EKF [9] fuses its output with wheel odometry to publish `odom -> base_footprint` (Fig. 7). Because there is no absolute heading, the EKF takes the IMU's yaw in relative mode, and yaw drift is corrected higher up, by slam_toolbox or AMCL through `map -> odom`. Exactly one of the two publishes that transform at any time (transform tree in Appendix, Fig. 16).

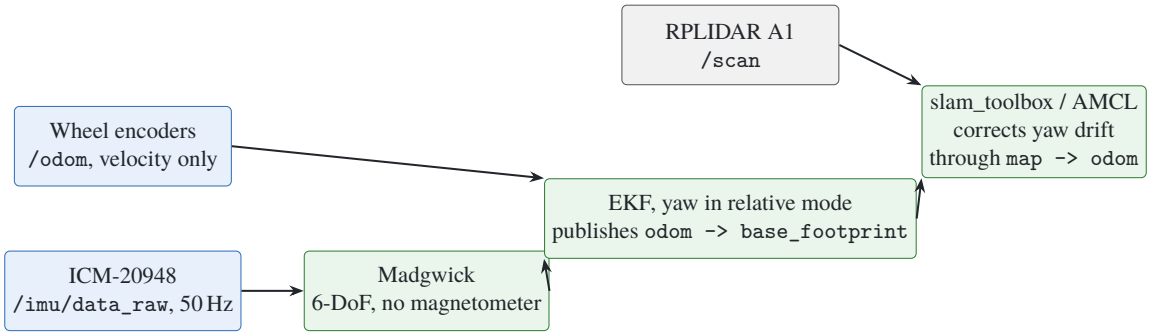



**Figure 7:** State estimation. With no magnetometer, the map corrects heading.

**Mapping and navigation.** slam_toolbox [3] builds maps from the RPLIDAR A1, used to 8 m. Nav2 [2] runs with AMCL on saved maps and the MPPI controller [12] in its omnidirectional (`Omni`) motion model, which lets the planner use the base's sideways motion. A velocity smoother and a collision monitor sit between the controller and the wheels (Appendix, Fig. 17). Nav2 is limited to 0.5 m/s on each of $x$ and $y$ and to 1.5 rad/s in rotation.

**Exploration.** A frontier explorer, a Python re-implementation of the breadth-first frontier search of m-explore-ros2 [13, 14], is written to send goals to Nav2, but has not been tested against a tuned Nav2.

## 8 Web Dashboard

A React and TypeScript dashboard (Fig. 8) shows the battery, the four motors, odometry, the IMU, the LiDAR, the live map and the camera. It drives the base with on-screen controls or the keyboard, sets Nav2 goals and tunes the PID gains.

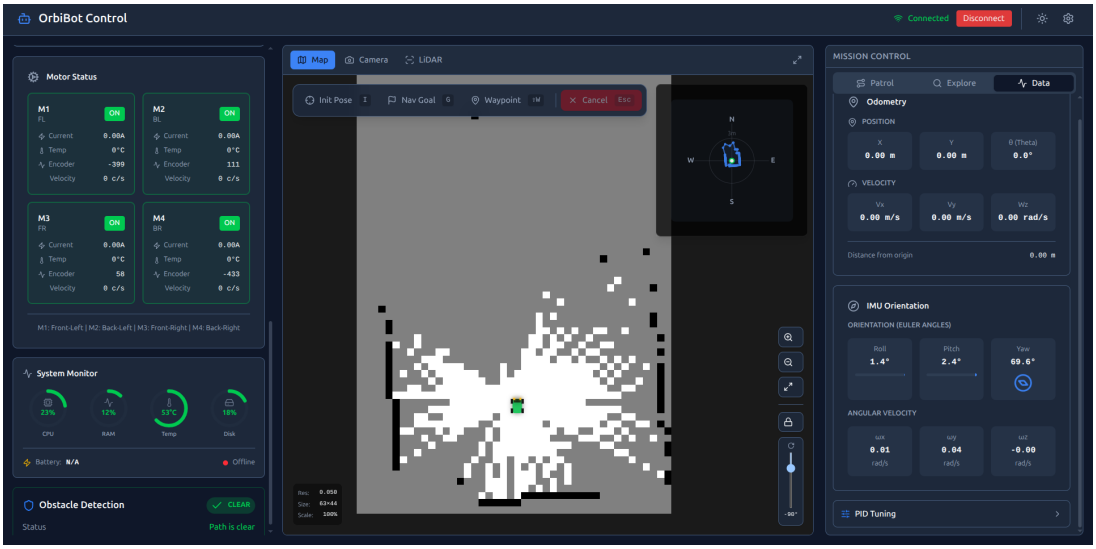



**Figure 8:** The dashboard: motor status, system monitor, the SLAM map, odometry and IMU orientation.

It reaches ROS 2 through rosbridge [19] as JSON over a WebSocket. The camera takes a separate path: a WebRTC server, itself a ROS 2 node, subscribes to the colour image topic and streams it to the browser as video, so camera frames never pass through rosbridge (Appendix, Fig. 18). The dashboard never drives hardware directly: its commands use the same topics as every other client.

Building it showed that the cost of a robot dashboard is not drawing, which happens in the operator's browser, but what the robot must serialise and send: each of about fifteen subscriptions is turned into JSON separately on the Pi. A cheaper design, with one packed telemetry message, a decimated scan, a map sent on request and a camera path that skips ROS 2 entirely, is drawn up but not built.

## 9 LLM Agent

`orbibot_agent` lets a language model operate the robot through tools (Fig. 9). It works with Google Gemini, Anthropic Claude, Moonshot Kimi, or a local model served by Ollama, behind one provider interface with streaming. An orchestrator sends the user's message and the tool schemas to the model, executes the tool calls it returns, and feeds the results back, for up to ten rounds (sequence in Appendix, Fig. 19).

**Tools.** The 58 tools fall into seven groups: motion (4), sensors (13), firmware (7), generic ROS 2 access (10), navigation and SLAM (19), diagnostics (3) and analysis (2). Each is a typed function with a JSON schema. The agent talks to ROS 2 directly through `rclpy`, not through rosbridge.

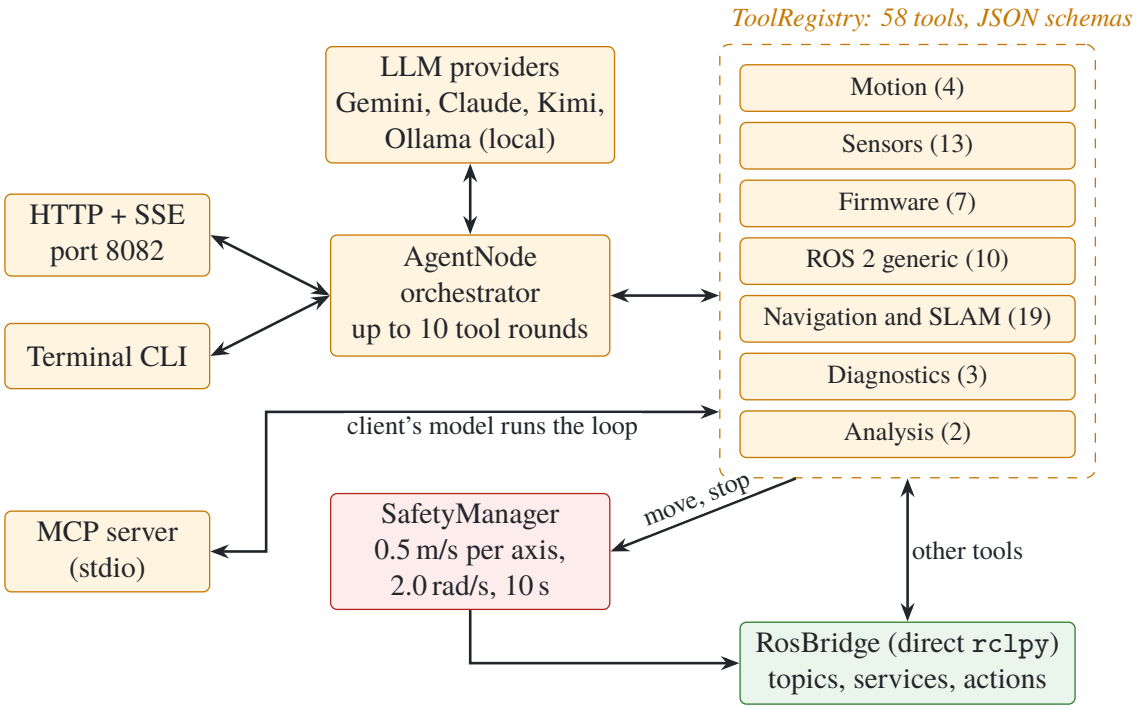



**Figure 9:** The agent. The HTTP and terminal interfaces share one orchestrator; the MCP server hands the tools straight to the client's own model. The 58 tools in seven groups reach ROS 2 directly through `rclpy`; the two direct motion tools pass the safety manager first.

**Interfaces.** An HTTP server with streamed replies, which the dashboard's chat panel uses, and a terminal client both run through the orchestrator. An MCP server over standard input and output exposes the tools directly, so that a general-purpose AI client, whose own model then runs the loop, can work with the robot. On 2026-03-14 the agent itself, driven through this MCP server, ran a baseline check of the robot's health, sensors and firmware, which is kept in the repository; its IMU figures predate the IMU fixes and are not used here.

**Safety.** The two direct motion tools, `move_robot` and `stop_robot`, pass a safety manager that clamps each linear axis to 0.5 m/s and rotation to 2.0 rad/s and caps any single motion at 10 s. It also checks an emergency-stop flag, but nothing in the code sets that flag yet.

Other tools can move the robot without the safety manager: the Nav2 tools, which fall under Nav2's own limits, and the generic topic-publishing tool, which can write any velocity to `/cmd_vel`. For all of them, the firmware's limits and timeout still apply (Fig. 10). Routing every motion through the safety manager is an open fix.

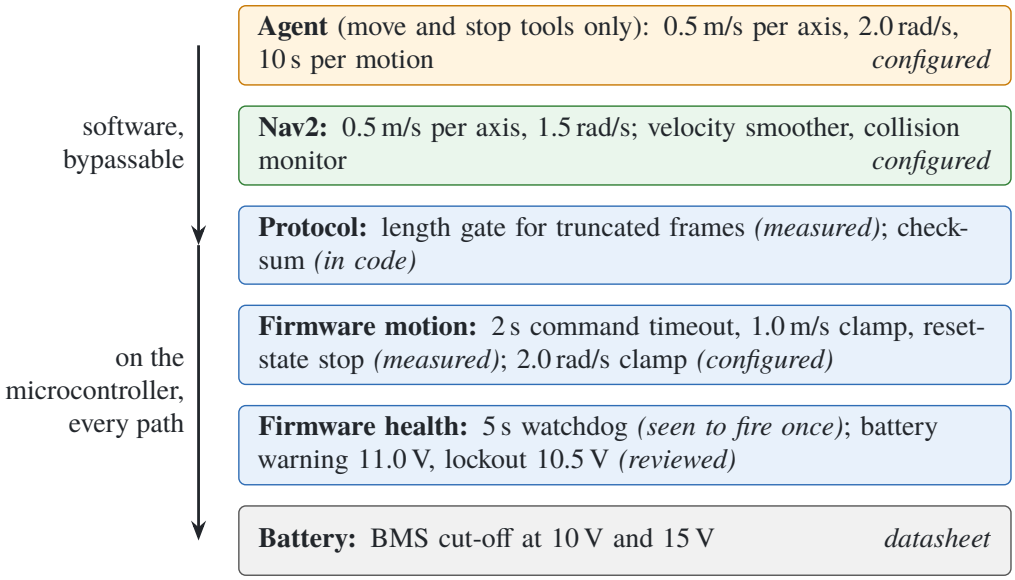



**Figure 10:** Safety layers, and how far each has been checked.

## 10 Bench Evaluation

All motion tests were run with the robot raised and the wheels free to spin, on 2026-04-19 and 2026-04-28. Test scripts on the host sent commands straight over the serial link and logged the firmware's reports; static sensor checks followed on 2026-08-03.

These tests check directions, signs, tracking and safety behaviour. They cannot check distance, drift or traction, which need the robot on the floor. The full procedures and logs are in the firmware test report [6].

### 10.1 Kinematics and velocity tracking

Each of four motions was commanded for 2 s, re-sent every 100 ms. All four wheel-direction patterns matched the inverse kinematics, so no motor or encoder was wired backwards.

Table 2 and Fig. 11 give the measured chassis velocity. Forward, backward and turning were within 4.1 % of the command; sideways motion reached 90.7 %, and the off-axis components stayed below 20 mm/s or mrad/s.

**Table 2:** Commanded and measured chassis velocity, 2 s runs, wheels off the ground (2026-04-19). Units: mm/s and mrad/s.

| Motion | Command | $v_x$ | $v_y$ | $\omega$ |
|---|---|---|---|---|
| Forward | $v_x = 150$ | 148 | 17 | −19 |
| Backward | $v_x = -150$ | −154 | 0 | 0 |
| Sideways | $v_y = 150$ | −5 | 136 | 19 |
| Turning | $\omega = 700$ | 0 | −11 | 729 |

In the same runs the back-right encoder counted about 9–16 % fewer ticks than the others. A later run

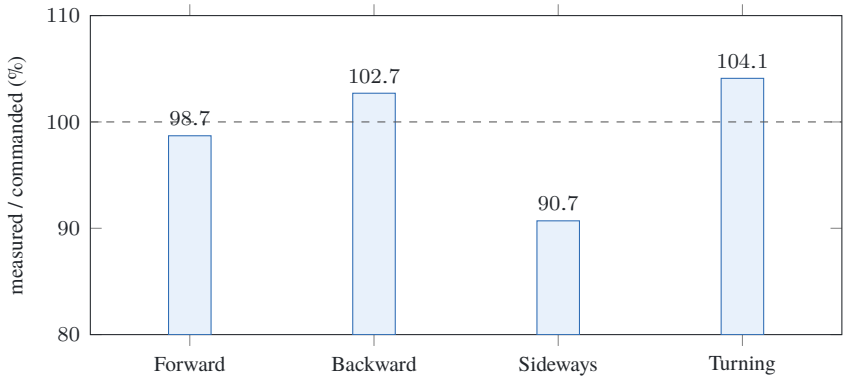



**Figure 11:** Measured chassis velocity as a share of the command (dashed line: 100 %).

at the 1.0 m/s limit, 5 s long with about 250 samples per motor, showed no weak motor at steady state: once the loops converged, the four wheels averaged 1001.7 to 1002.5 mm/s, 0.1 % apart (Fig. 12). The test report attributes the earlier gap to start-up stiction in that drivetrain.

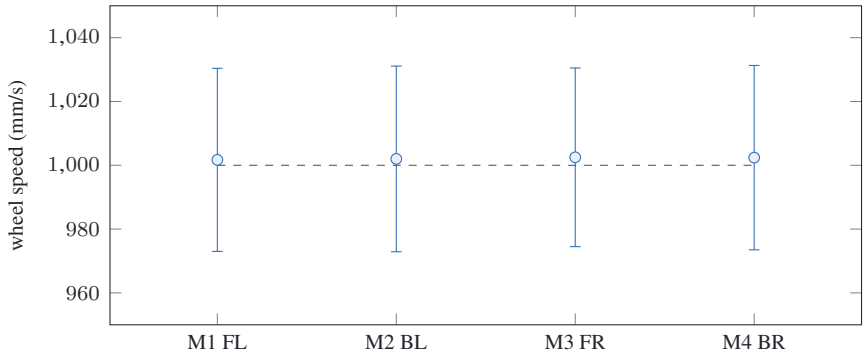



**Figure 12:** Mean wheel speed at the 1.0 m/s limit, with one standard deviation (2026-04-28).

## 10.2 Safety behaviour

A single command of 0.1 m/s was sent and nothing after it. The firmware reported motion until 1.82 s and zero from 2.12 s on (Fig. 13). With reports every 0.3 s in this log, the stop falls between those two samples, on the 2 s setting.

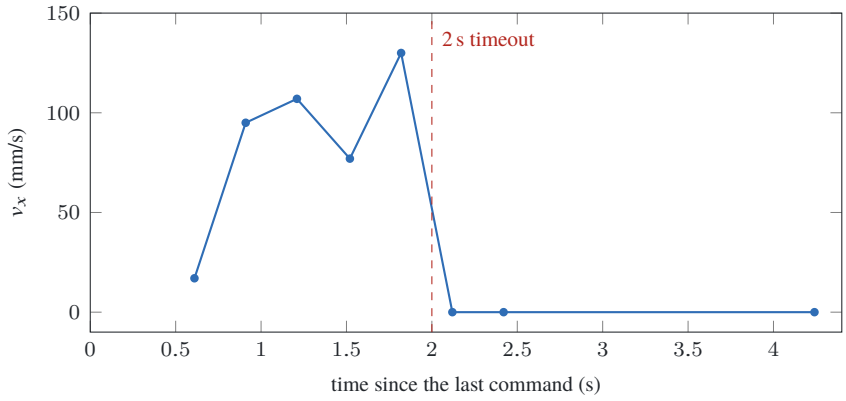



**Figure 13:** Command timeout: one 0.1 m/s command at $t = 0$, then silence (2026-04-19).

A command of 3 m/s, three times the limit, produced a peak of 1023 mm/s; the 2.0 rad/s rotation clamp

was not exercised. A reset-state command during motion stopped the motors at once. The watchdog fired once, by accident, during the malformed-input test described below, and reset the board as designed. The battery lockout was reviewed in the code but not triggered, since that would mean draining the battery.

## 10.3 Serial link and robustness

With all five reports enabled the firmware sent 270 frames per second, 54 Hz per type against a 50 Hz target; the difference comes from the scheduler's tick granularity and is the same for all five streams. Commands were then sent at a target of 500 Hz, of which the host achieved about 328 Hz (1083 in 3.30 s), while 825 reports were parsed, with no checksum error, overflow or torn frame. An unknown function code was dropped without interrupting the stream.

**A critical fault, found and fixed**

A frame with a valid checksum but no payload made the firmware act on stale data in memory:

- a truncated motion command drove the wheels at 1029 mm/s;
- a truncated PID command caused a 993 mm/s spike;
- a truncated calibration command froze the reports until the watchdog reset the board.

A per-function minimum length check, added before any handler runs, fixed all three. After it, twenty truncated frames of each of five kinds caused no motion and no gap in the reports.

## 10.4 Sensors, rates and resources

With the robot level and still, the accelerometer read $|a| = 9.802\,\mathrm{m/s^2}$, 0.9995 of standard gravity, after a fix to its Z axis on 2026-08-03; X and Y read 14 and 16 $\mathrm{mm/s^2}$. The gyro's Z axis showed a bias of $-15$ mrad/s (0.86 °/s) in April. On the same day as the accelerometer fix, the full chain from firmware to ROS 2 delivered `/imu/data_raw`, `/joint_states` and `/orbibot/encoders` at about 50 Hz.

## 10.5 What was run but not measured

Several parts of the system were used on the robot but produced no numbers:

- **on video:** maps built with slam_toolbox while driving (from July 2025), localising and driving on a saved map of a home (January 2026), and driving from the dashboard with the live camera and map (February 2026);
- **in the project's status notes only:** Nav2 set up and run in a small room (March 2026), and the agent running on the robot behind the dashboard.

They show that the pieces work together; they do not show how well.

## 11 Limitations

**Nothing measured on the floor.** Distance, drift, traction, PID under load, Nav2's success rate and path quality, and the agent's task success are all unmeasured. The 1.74 m/s top speed is worked out from the motor datasheet; the firmware caps the robot at 1.0 m/s.

**No command arbitration.** The dashboard, the agent and Nav2's collision monitor all publish `/cmd_vel` (the switched-off gamepad driver is still wired to it), and the last writer wins. A Nav2 goal and the dashboard's joystick can fight mid-drive, and closing the dashboard mid-command leaves the last velocity in force until the firmware's 2 s timeout. A fix with `twist_mux`, priorities and per-source timeouts is designed but not built (Appendix, Fig. 20).

**Agent safety has gaps.** Only the agent's two direct motion tools pass its safety manager; its Nav2 tools and its generic topic-publishing tool do not, and its emergency-stop flag is never set. The firmware's clamp and timeout are the only limits common to every path.

**Estimation data before August 2026 is suspect.** Two faults fed the estimator bad data for months: the accelerometer read too high on Z, and raw IMU packets were silently dropped by a length check whenever the firmware and driver were out of step. Any tuning or drift figure recorded before 2026-08-03 should be measured again; the EKF must be re-tuned.

**No absolute heading.** Without the magnetometer, yaw comes from the gyro alone between map corrections; its drift rate has not been measured.

**Very low speeds.** Below about 50 mm/s per wheel the fixed dead-band offset makes the wheels uneven: at 30 mm/s for 2.5 s the encoders counted 54, 351, 407 and 0 ticks against about 315 expected (the firmware test report first gave 158, an arithmetic slip since corrected). Navigation speeds are unaffected, but a planner's slow final approach could be.

**Other limits.** The accelerometer's X and Y scale has not been checked under a known acceleration; storing PID gains in flash across a reboot was not exercised; the depth camera ran on USB 2; and Nav2's footprint and inflation were never measured.

## 12 Lessons Learned

**Put deadlines on the microcontroller.** Keeping every timed loop and every stop condition in the firmware meant each could be tested on its own, from a script, with no ROS 2 stack running; whether the split also holds under heavy host load is still to be measured.

**Test what the parser does with bad input.** The most dangerous fault found was not in control but in parsing: a well-formed but empty frame made the wheels move.

**A 2D LiDAR sees one slice.** A table top above the scan plane or a low box below it is invisible to the map, which is why the depth camera was added.

**Write down what was measured.** Recording each test with its date, conditions and limits is what made it possible to see that months of estimator tuning rested on faulty IMU data, and to say so here.

## 13 Conclusion and Future Work

OrbiBot V1 shows that a hand-built mecanum robot on a Raspberry Pi 5 and a commercial STM32 board can carry the full ROS 2 navigation stack and an LLM agent, if the microcontroller keeps every deadline and enforces its own safety limits. The bench tests confirm the firmware's kinematics, tracking, linear speed limit and timeout, and found and fixed one critical parsing fault.

The measurements that would close the gaps in Section 11 are floor tests: odometry drift on closed square paths, Nav2's success rate and time over a fixed set of goals, the agent's success rate on a fixed set of natural-language tasks across model providers, and wheel behaviour under heavy host load. Command arbitration, and routing all of the agent's motion through its safety manager, would come before any of them.

## Acknowledgements

The firmware keeps a protocol compatible with Yahboom's, and the serial driver started from Yahboom's library. The firmware uses FreeRTOS, ST's STM32F10x Standard Peripheral Library, ARM's CMSIS and the font table of stm32-ssd1306 by Aleksander Alekseev, with their notices kept; its SSD1306 display routines follow an existing driver's interface, whose source is still being traced. The dashboard uses shadcn/ui components (MIT). Three wheel and LiDAR meshes in the robot model are third-party CAD whose source was not recorded. Design ideas came from LGDXRobot2 and AuBot.

Parts of the firmware audit, its bench tests and this documentation were carried out with AI coding assistants under the author's direction; every result reported here was checked against the logged data and the code.

## References

[1] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot operating system 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022.

[2] S. Macenski, F. Martín, R. White, and J. Ginés Clavero, "The marathon 2: A navigation system," in *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020, pp. 2718–2725.

[3] S. Macenski and I. Jambrecic, "SLAM Toolbox: SLAM for the dynamic world," *Journal of Open Source Software*, vol. 6, no. 61, p. 2783, 2021.

[4] S. H. Vemprala, R. Bonatti, A. Bucker, and A. Kapoor, "ChatGPT for robotics: Design principles and model abilities," *IEEE Access*, vol. 12, pp. 55 682–55 696, 2024.

[5] Anthropic, "Model context protocol specification," https://modelcontextprotocol.io, 2024, accessed October 2026.

[6] J. Nath, "OrbiBot V1: Source code, firmware, documentation and test reports," https://github.com/jewel-nath/orbibot-v1, 2026, Apache-2.0.

[7] LGDXRobot2, "LGDXRobot2: An open mecanum-wheel robot," https://lgdxrobot.uk, accessed October 2026.

[8] P. F. Muir and C. P. Neuman, "Kinematic modeling for feedback control of an omnidirectional wheeled mobile robot," in *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, 1987, pp. 1772–1778.

[9] T. Moore and D. Stouch, "A generalized extended Kalman filter implementation for the Robot Operating System," in *Proc. 13th International Conference on Intelligent Autonomous Systems (IAS-13)*. Springer, 2014.

[10] S. O. H. Madgwick, "An efficient orientation filter for inertial and inertial/magnetic sensor arrays," University of Bristol, Tech. Rep., 2010.

[11] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. MIT Press, 2005.

[12] G. Williams, N. Wagener, B. Goldfain, P. Drews, J. M. Rehg, B. Boots, and E. A. Theodorou, "Information theoretic MPC for model-based reinforcement learning," in *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, 2017, pp. 1714–1721.

[13] B. Yamauchi, "A frontier-based approach for autonomous exploration," in *Proc. IEEE International Symposium on Computational Intelligence in Robotics and Automation (CIRA)*, 1997, pp. 146–151.

[14] robo-friends, "m-explore-ros2: Frontier exploration for ROS 2," https://github.com/robo-friends/m-explore-ros2, BSD licence; accessed October 2026.

[15] M. Ahn *et al.*, "Do as I can, not as I say: Grounding language in robotic affordances," in *Proc. Conference on Robot Learning (CoRL)*, 2022, arXiv:2204.01691.

[16] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng, "Code as policies: Language model programs for embodied control," in *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, 2023, pp. 9493–9500.

[17] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[18] FreeRTOS, "FreeRTOS real-time operating system kernel," https://www.freertos.org, MIT licence; accessed October 2026.

[19] C. Crick, G. Jay, S. Osentoski, B. Pitzer, and O. C. Jenkins, "Rosbridge: ROS for non-ROS users," in *Robotics Research: The 15th International Symposium ISRR*, ser. Springer Tracts in Advanced Robotics, vol. 100. Springer, 2016, pp. 493–504.

# A Additional Diagrams

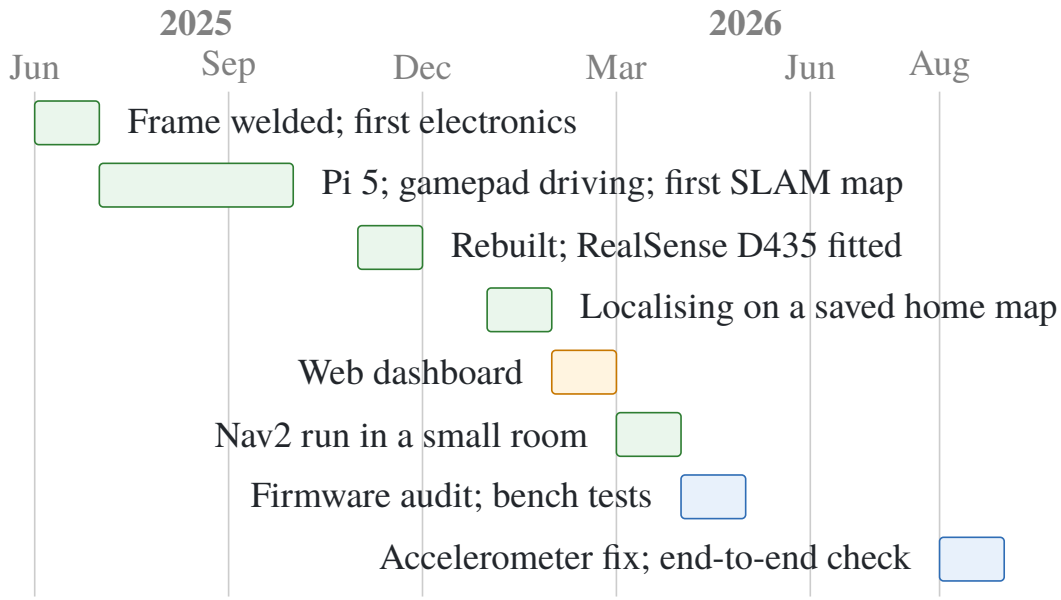
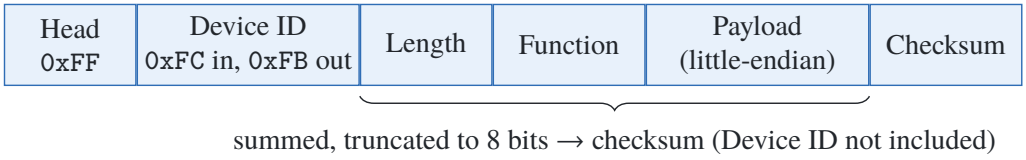



**Figure 14:** Build timeline, from the dated photos, videos, test reports and status notes. Green: robot and ROS 2; orange: dashboard; blue: firmware.




```
request:  FF FC 03 51 54
reply:    FF FB 04 51 02 00 57  (firmware 2.0)
```

**Figure 15:** Serial frame, with a version request and its reply as recorded on the robot.

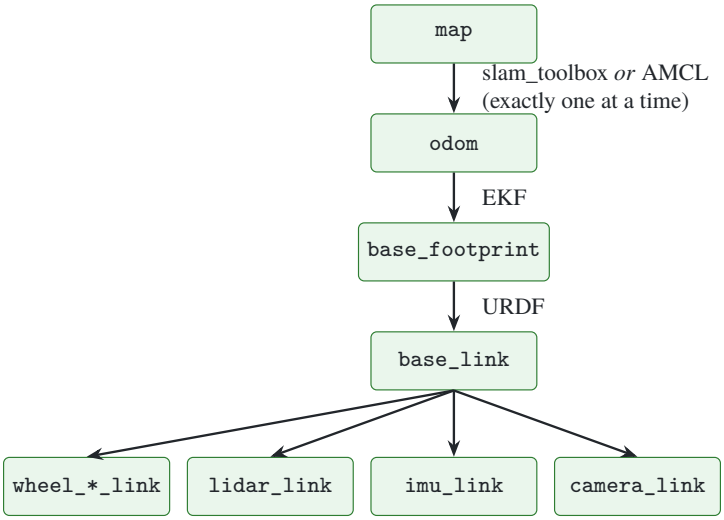



**Figure 16:** Transform tree; the base frame is `base_footprint`.

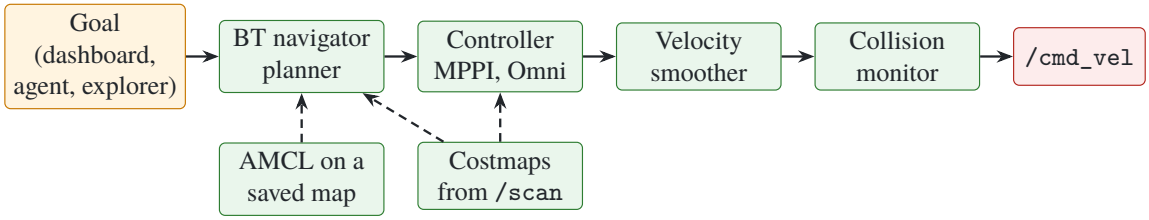



**Figure 17:** Nav2's command chain on V1.

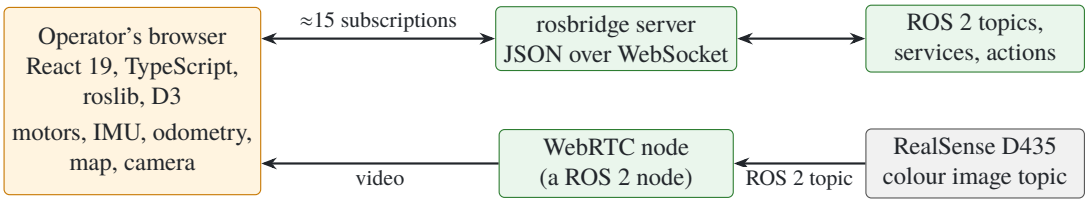



**Figure 18:** Dashboard data paths as built.

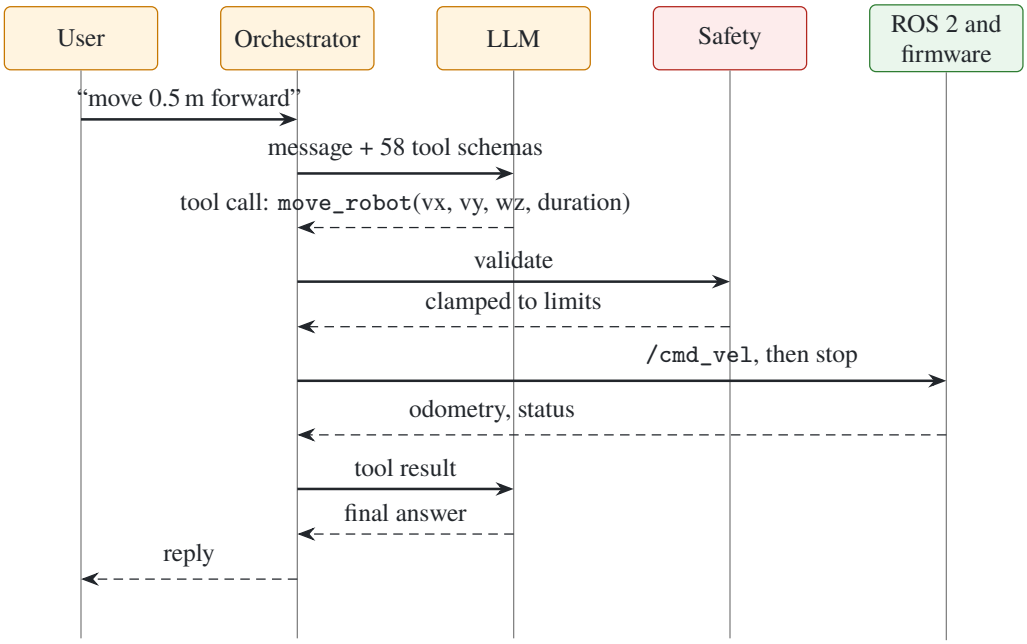



**Figure 19:** One request through the agent.

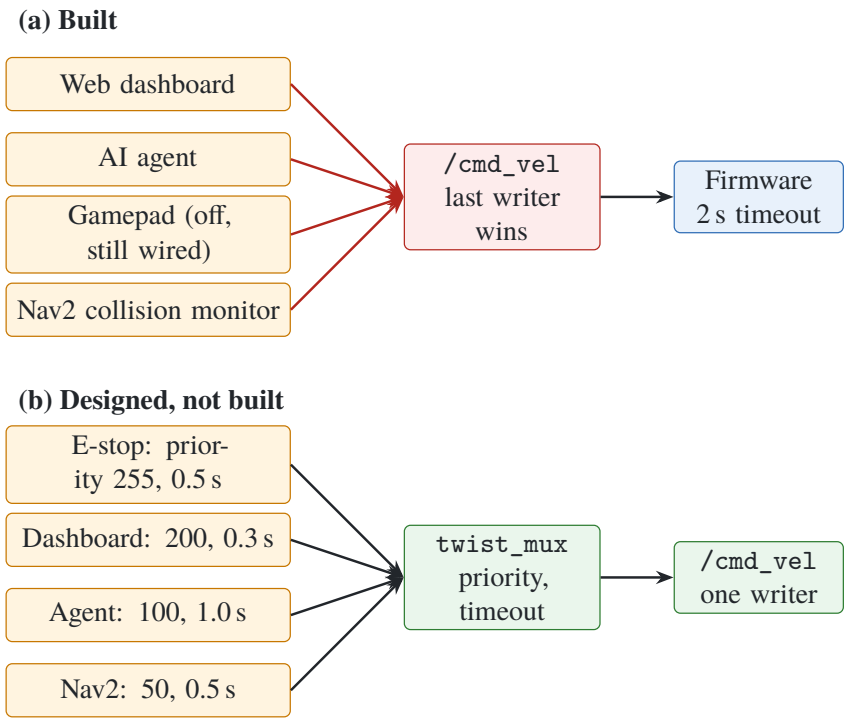



**Figure 20:** Velocity commands: (a) as built; (b) the designed fix, with priority and timeout per source.